ITDBADM

(REVIEW FOR MIDTERMS)

For today's session

- **General Review of Midterm Content**
  - Multiple Choice Questions
  - Essay Questions
  - Tasks: SQL Statements + Screenshots or Line Explanation
- **Midterms Rules**
  - Face-to-face class **(FEB 27, FRIDAY) GK304B**
  - Laptops / Tablets will **NOT** be allowed. Cheating of any form will not be tolerated.
  - We will mainly use **Sakila** and the **World** Database (Make sure these databases are uploaded correctly and tables are NOT empty. Delete all the Stored Procedures there).
  - The midterm exam will **START at 9:25 AM and END at 10:45 AM (80 MINUTES)**

Midterms Scope

- SQL Optimization Techniques
- DBA Role Definition
- MySQL Workbench Interface
- Big 6 SQL Statements
- Join Types
- Create Views
- Creating and Using Schemas
- Creating and Modifying Tables (INSERT, UPDATE, DELETE)
- Aggregation Functions
- Stored Procedures (IN and INOUT PARAMETERS)

Create a View (Summary of Daily Orders)

```
-- Create a view to show total sales per day
CREATE VIEW  daily_orders AS  -- "AS" means: define the view USING the query below
SELECT customer_name, item_name, quantity
FROM jollibee_orders.orders;

-- Generate the report of the created “View”
SELECT * FROM daily_orders;

-- Delete the “View” if no longer needed
DROP VIEW IF EXISTS daily_orders;
```

SQL OPTIMIZATION TECHNIQUES

- **Goal:** Make SQL queries run faster and use fewer resources. Optimization helps databases find and retrieve data efficiently by reducing unnecessary work.
- **Key ideas:**
  - Use indexes wisely to speed up lookups.
  - Avoid fetching more data than needed.
  - Check how the database runs your query (EXPLAIN).
  - Simplify nested queries when possible.
  - Limit and partition large datasets.
  - *In short: The cleaner and more specific your query, the faster it runs.*

SQL OPTIMIZATION TECHNIQUES

Technique	Benefit	Notes
Proper Indexing	Speeds up lookups, avoids full table scans	Use on frequently queried columns; composite indexes when filtering on multiple columns
Avoid SELECT *	Reduces data transfer and unnecessary I/O	Select only the columns you need
Rewrite subqueries / avoid nested queries	Can flatten operations, let optimizer do better joins	Use joins or derived tables instead of deep nesting
Limit result sets	Reduces unnecessary rows fetched	Use LIMIT when you don’t need all rows
Partitioning large tables	Helps MySQL scan only relevant partitions	Use when table is huge and queries often filter by partition column
Use efficient data types	Saves memory and improves performance	Choose smallest data type that fits your data

DBA Role

- A **Database Administrator (DBA)** is responsible for managing, maintaining, and securing an organization's databases to ensure optimal performance and data integrity.
- **DBA Role:**
  - Create Schemas and Tables
  - Load Large Datasets into the database
  - Use automation to keep things running smoothly
  - Tackle security, backup, and data recovery
  - Create stored procedures and triggers
  - Tune queries, design indexes, and monitor performance
  - Sometimes the role overlaps or collaborates with a DB Engineer and a Data Analyst

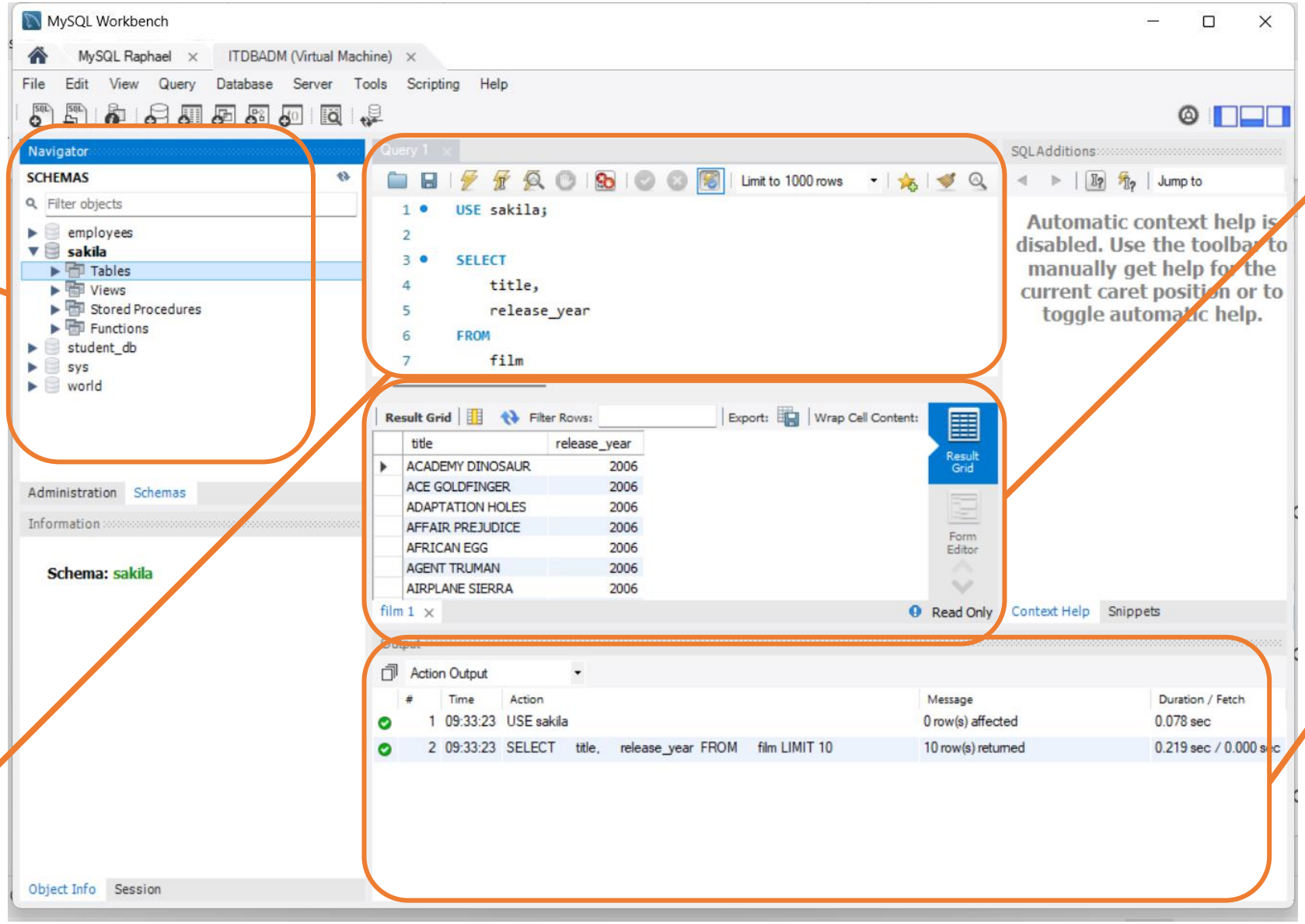
What does a DBA do?

Focus	DBA	Data Analyst
Focus area	Keeps databases running smoothly, securely, and efficiently	Turns raw data into insights for effective business decision-making
Daily Work	Manages databases, enhances security, optimizes performance, and prevents data loss	Analyzes trends, build reports, and works with teams to improve business performance
Tools	Works with database systems like MySQL and Cloud platforms to store and protect data	Power Bi, Tableau, SQL, Python, and Excel to analyze data
Business Impact	Assures that data is safe, reliable, and readily available so business can operate without disruption	Helps businesses spot opportunities, reduce risk, and grow through data-driven strategies

MySQL Workbench Interface

Schemas Tab:
You can see your tables, views, and functions in your schemas

Query Editor Window:
This is where you write and run you code



Result Grid:
This shows the output of your SQL queries in a table format

Action Output:
A summary of actions taken by the server.

Note: The Message collum is useful for troubleshooting

Basic Elements "Big 6" of a SQL SELECT Statement

Start

SELECT	**SELECT** columnName	***Identifies the column(s) you want your query to select for your results.***
FROM	**FROM** tableName	***Identifies the table(s) your query will pull data from***
WHERE	**WHERE** logicalCondition	***(Optional) Specifies record-filtering criteria for filtering your results***
GROUP BY	**GROUP** BY columnName	***(Optional) Specifies how to group the data in your results***
HAVING	**HAVING** logicalCondition	***(Optional) Specifies group-filtering criteria for filtering your results***
ORDER BY	**ORDER BY** columnName	***(Optional) Specifies the order in which your query results are displayed***

End

Common JOIN types

INNER JOIN	**FROM** leftTableName **INNER JOIN** rightTableName	*Returns records that exist in **BOTH** tables, and **EXCLUDES** unmatched records from either table*
LEFT JOIN	**FROM** leftTableName **LEFT JOIN** rightTableName	*Returns ALL records from the **LEFT** table, and any matching records from the **RIGHT** table*
RIGHT JOIN	**FROM** leftTableName **RIGHT JOIN** rightTableName	*Returns ALL records from the **RIGHT** table, and any matching records from the **LEFT** table*
FULL OUTER JOIN	**FROM** leftTableName **FULL JOIN** rightTableName	*Returns ALL records from **BOTH** tables, including **NON-MATCHING** records*
UNION	**SELECT FROM** firstTableName **UNION** **SELECT FROM** secondTableName	*Returns all data from one table, with all data from another table **APPENDEDED TO THE END*** ** You can also use **UNION ALL** to keep duplicates*

CREATE SCHEMA

- Used to create a new table in the database
- Syntax:

```
CREATE SCHEMA schema_name;
```

```
USE schema_name;
```

- Example:

```
CREATE SCHEMA pbb_collab;
USE pbb_collab;
```

```
CREATE SCHEMA sakila;
USE sakila;
```

Creating a table

```
-- Sample of a general table with different data types
CREATE TABLE table_name (
    Housemate_ID INT PRIMARY KEY AUTO_INCREMENT,        -- Primary Key
    FullName VARCHAR(100),                              -- Variable-length string (up to 100 chars)
    Age INT,                                            -- Integer. Number
    Height DECIMAL(5,2),                                -- Decimal (e.g., 170.55 cm)
    Gender ENUM('Male', 'Female', 'Other'),             -- Predefined data types.
    Eviction_Status BOOLEAN,                            -- Boolean (TRUE/FALSE)
    BirthDate DATE,                                     -- Date (YYYY-MM-DD)
    RegistrationTimestamp DATETIME                      -- Date and time Eg. '2025-06-08 14:30:00'
);
```

MAKE SURE YOU KNOW THE SQL SAFE UPDATES!

INSERT ROW

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

DELETE ROW

```
DELETE FROM table_name
WHERE condition;
```

INSERT COLLUMN

```
ALTER TABLE table_name
ADD COLUMN column_name data_type;
```

DELETE COLLUMN

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

Practice Prompts (Sakila Database)

STRING/TEXT FUNCTIONS

Task: Display the full names of all customers in LOWERCASE format and show how many characters each full name has.

Tables Used: customer

Expected Output Columns: customer_id, full_name, name_length

Statements Used: CONCAT(), LOWER(), LENGTH()

```
SELECT customer_id,
    LOWER(CONCAT(first_name, ' ', last_name)) AS full_name,
    LENGTH(CONCAT(first_name, ' ', last_name)) AS name_length
FROM customer;
```

customer_id	store_id	first_name	last_name
1	1	MARY	SMITH
2	1	PATRICIA	JOHNSON
3	1	LINDA	WILLIAMS
4	2	BARBARA	JONES
5	1	ELIZABETH	BROWN

customer_id	full_name	name_length
1	mary smith	10
2	patricia johnson	16
3	linda williams	14
4	barbara jones	13
5	elizabeth brown	15

Practice Prompts (Sakila Database)

Table: payment

payment_id	customer_id	staff_id	rental_id	amount	payment_date	last_update
1	1	1	76	2.99	2005-05-25 11:30:37	2006-02-15 22:12:30
2	1	1	573	0.99	2005-05-28 10:35:23	2006-02-15 22:12:30
3	1	1	1185	5.99	2005-06-15 00:54:12	2006-02-15 22:12:30
4	1	2	1422	0.99	2005-06-15 18:02:53	2006-02-15 22:12:30
5	1	2	1476	9.99	2005-06-15 21:08:46	2006-02-15 22:12:30
6	1	1	1725	4.99	2005-06-16 15:18:57	2006-02-15 22:12:30

AGGREGATE FUNCTIONS (MIN, MAX, COUNT, SUM, AVG)

MIN() → Finding the lowest number/smallest value
MAX() → Finding the highest number/largest vlaue

```
SELECT
   MIN(amount) AS lowest_profit,
   MAX(amount) AS highest_profit
FROM payment;
```

MIN MAX

lowest_profit	highest_profit
0.00	11.99

COUNT() → adding all the rows in a specific column
SUM() → adding all the values together
AVG() → calculate the average values

```
SELECT
   COUNT(amount) AS total_transactions,
   SUM(amount) AS total_profit,
   AVG(amount) AS average_profit
FROM payment;
```

COUNT SUM AVG

total_transactions	total_profit	average_profit
16044	67406.56	4.201356

Practice Prompts (Sakila Database)

AGGREGATE FUNCTIONS (MIN, MAX, COUNT, SUM, AVG)

Task 1: Count the Top 5 Customer Rentals
Tables Used: customer, rental
Expected Output Columns: customer_id, first_name, last_name, total_rentals
Statements Used: JOIN, COUNT(), ORDER BY, LIMIT

Task 2: Find the Highest Payment Made By Each Customer
Tables Used: payment, customer
Expected Output Columns: customer_id, first_name, last_name, max_payment
Statements Used: JOIN, MAX(), GROUP BY

Task 3: Show the duration (in days) for each rental, and sort to show the shortest ones first.
Tables Used: rental
Expected Output Columns: rental_id, rental_date, return_date, duration_days
Statements Used: DATEDIFF(), IS NOT NULL, ORDER BY

Task 4: Total Revenue per Store
Tables Used: payment, staff, store
Expected Output Columns: store_id, total_revenue
Statements Used: JOIN, SUM(), GROUP BY

Task 5: Average Payment Amount per Film
Tables Used: payment, rental, inventory, film
Expected Output Columns: film_id, title, average_payment
Statements Used: JOIN, AVG(), GROUP BY

Task 6: Find customers who have at least one rental that hasn't been returned yet. (Tip: return_date is null)
Tables Used: customer, rental
Expected Output Columns: customer_id, first_name, last_name, overdue_count
Statements Used: JOIN, IS NULL, COUNT(), GROUP BY, HAVING

Task 7: Display the total payment amount per month to identify revenue trends.
Tables Used: payment
Expected Output Columns: month_name, total_revenue
Statements Used: MONTH(), SUM(), GROUP BY

Basic Syntax of a Stored Procedure

```
DELIMITER //
CREATE PROCEDURE procedure_name()
BEGIN
   -- SQL statements go here
END;
// DELIMITER ;
```

```
DELIMITER $$
CREATE PROCEDURE procedure_name()
BEGIN
   -- SQL statements go here
END;
$$ DELIMITER ;
```

A **delimiter** is a character (or set of characters) that tells MySQL where **one SQL statement ends** and another begins.

Tips:

- Always **reset** the delimiter back to ; at the end.
- Avoid symbols that may already have meaning in your queries (e.g., : or &).
- In command-line clients (e.g., MySQL CLI), simpler delimiters like **//** or **$$** are safer.
- In **MySQL Workbench**, **$$** is often the default delimiter.

Stored Procedures (Parameters)

Parameter Type	Meaning	Direction	Can Modify?
IN	Input only	You send a value into the procedure	Cannot be changed inside
OUT	Output only	Procedure sends a result back	You receive a value
INOUT	Input + Output	You send a value, and get back a new one	Can be modified and returned

Practice (Sakila DB)

FIND FILMS BY A SPECIFIC RATING

Task: List all films with a specific movie rating provided by the user.

Tip: Use Stored Procedures, IN parameter

Input data: Rating (e.g., 'PG', 'G', 'R')

Tables Used: film

Expected Output Columns: film_id, title, rating

Statements Used: SELECT, WHERE

Call the Procedure: Show the films that are rated 'PG', 'R', 'NC-17'

Rating Types:

G	(General Audiences)
PG	(Parental Guidance)
PG-13	(Parents Strongly Cautioned)
R	(Restricted)
NC-17	(Adults Only)

```
DELIMITER $$
CREATE PROCEDURE GetFilmsByRating(
   IN rating_input VARCHAR(5))
BEGIN
   SELECT film_id, title, rating
   FROM film
   WHERE rating = rating_input;
END
$$ DELIMITER ;
```

```
CALL GetFilmsByRating('PG');
```

Practice (Sakila DB)

Shortest Film	Longest Film	Average Length
46 minutes	185 minutes	115 minutes

FIND FILMS LONGER THAN A GIVEN LENGTH

Task: List all films that are longer than a given number of minutes.

Tip: Use Stored Procedures, IN parameter

Input data: Minimum film length

Tables Used: film

Expected Output Columns: film_id, title, length

Statements Used: SELECT, WHERE

Call the Procedure:

- Show the films longer than 120 minutes
- Show the films shorten than 60 minutes

```
DELIMITER $$
CREATE PROCEDURE GetFilmsByMinLength(
   IN min_length INT
)
BEGIN
   SELECT film_id, title, length
   FROM film
   WHERE length > min_length;
END
$$ DELIMITER ;
```

```
CALL GetFilmsByMinLength(120);
```

DECLARE AND SET VARIABLES

- What is **DECLARE**?
  - Used **inside** a stored procedure to **create a local variable**.

```
DECLARE variable_name datatype;
```

- What is **SET**?
  - Used to **assign a value** to a variable.

```
SET variable_name = value;
```

- Example. Declaring a variable to count the number of rental transactions:

```
DECLARE rentalCount INT;
SET rentalCount = 5;
```

COUNT(*) **INTO** rentalCount;

Practice (Sakila DB)

COUNT RENTALS OF A SPECIFIC CUSTOMER
Task: Count the total number of rentals made by a customer and return the count using an **INOUT** parameter.
Tip: Use Stored Procedures, INOUT parameter
Input data: Customer ID

Tables Used: rental (The rental table already has the customer id).
Expected Output Columns: rental count (returned via INOUT)
Statements Used: JOIN, COUNT(), DECLARE, SET
Call the Procedure: Show the total number of rentals for customer ID 12, 50, 80

```
-- Create the procedure
DELIMITER $$
CREATE PROCEDURE CountRentalsByCustomer(
    INOUT customerID INT
)
BEGIN
    DECLARE rentalCount INT;
    SELECT COUNT(*) INTO rentalCount
    FROM rental
    WHERE customer_id = customerID;
    SET customerID = rentalCount;
END
$$ DELIMITER ;
```

```
-- Declare a variable
SET @cust_id = 12;

-- Call the procedure
CALL CountRentalsByCustomer(@cust_id);

-- View the result
SELECT @cust_id AS rental_count;
```

Alternative Solution Using IN and OUT

COUNT RENTALS OF A SPECIFIC CUSTOMER

Task: Count the total number of rentals made by a customer and return the count using IN and OUT parameters.

Tip: Use Stored Procedures, IN and OUT parameters
Input data: Customer ID

Tables Used: rental, customer
Expected Output Columns: rental count (returned via INOUT)
Statements Used: JOIN, COUNT(), DECLARE, SET
Call the Procedure: Show the total number of rentals for customer ID 12, 60, 90

```
-- Create the procedure
DELIMITER $$
CREATE PROCEDURE CountRentalsByCustomer(
   IN customerID INT,
   OUT rentalCount INT)
BEGIN
   SELECT COUNT(*) INTO rentalCount
   FROM rental
   WHERE customer_id = customerID;
END
$$ DELIMITER ;
```

```
-- Call the procedure
CALL CountRentalsByCustomer(12, @result);

-- View the results
SELECT @result;
```

Practice (Sakila DB)

CALCULATE TOTAL PAYMENT OF A CUSTOMER

Task: Calculate the total amount paid by a customer and return the value through an INOUT parameter.
Tip: Use Stored Procedures, INOUT parameter
Input data: Customer ID

Tables Used: payment
Expected Output Columns: total payment amount (returned via INOUT)

Total Payment Data Type: DECIMAL(10,2)
Statements Used: SUM(), WHERE, DECLARE, SET

Call the Procedure: Show the total payment made by customer ID 7

```
-- Create the procedure
DELIMITER $$
CREATE PROCEDURE GetTotalPaymentByCustomer(
   INOUT customerID INT)
BEGIN
   DECLARE totalPaid DECIMAL(10,2);
   SELECT SUM(amount) INTO totalPaid
   FROM payment
   WHERE customer_id = customerID;
   SET customerID = totalPaid;
END
$$ DELIMITER ;
```

```
-- Declare a variable
SET @cust_id = 7;

-- Call the procedure
CALL GetTotalPaymentByCustomer(@cust_id);

-- View the result
SELECT @cust_id AS total_payment;
```

Practice (World DB)

COUNT COUNTRIES IN A CONTINENT
Task: Count the number of countries in a given continent and return the value using INOUT.
Tip: Use Stored Procedures, INOUT parameter
Input data: Continent Name

Tables Used: country
Expected Output Columns: count (via INOUT)
Statements Used: COUNT(), WHERE, SET
Call the Procedure: Show the number of countries in **Asia, Europe, and Africa**

```
-- Create the Stored Procedure
DELIMITER $$
CREATE PROCEDURE CountCountriesByContinent(
   INOUT continentName VARCHAR(50))
BEGIN
   DECLARE countryCount INT;
   SELECT COUNT(*) INTO countryCount
   FROM country
   WHERE continent = continentName;
   SET continentName = countryCount;
END
$$ DELIMITER ;
```

```
-- Declare the INOUT variable
SET @continent = 'Asia';

-- Call the procedure
CALL CountCountriesByContinent(@continent);

-- Check the result
SELECT @continent AS country_count;
```

REVIEW OF STORED PROCEDURES

INSTRUCTIONS:

1. Use MySQL Workbench to write and run your stored procedure.

- ***Sakila*** or ***World*** database will be your data source.
- The stored procedure might use either IN or INOUT parameters.

2. Copy your full SQL code

- It should include Stored Procedures (DELIMITER, CREATE PROCEDURE, CALL, and/or SELECT statements)
- Paste the SQL code to Canvas as part of your submission.
- You may also save all the codes into a txt file.

3. Show the output results as well

- Take a screenshot of the result table(s) produced when you call the procedure.
- You may also copy-paste the result table output if you don't' want to screenshot

Task 1

```
DELIMITER $$
CREATE PROCEDURE GetRentalsByCustomer(IN cust_id INT)
BEGIN
SELECT r.rental_id, r.customer_id, r.rental_date
FROM rental AS r
WHERE r.customer_id = cust_id;
END $$
DELIMITER ;
CALL GetRentalsByCustomer(35);
CALL GetRentalsByCustomer(66);
CALL GetRentalsByCustomer(78);
```

EACH CUSTOMER RENTAL TRANSACTION

Task 1: List all rentals made by a specific customer.

Database: sakila

Tip: Use Stored Procedures, **IN** parameter

Input data: Customer ID

Tables Used: rental (Tip: Customer ID is already in the rental table)

Expected Output Columns: rental_id, customer_id, rental_date

Statements Used: JOIN, WHERE

Call the Procedure: Show the rental records for the customer with

ID 35, 66, 78 separately

Task 2

VERIFY STORE INVENTORY COUNT

Task 2: Count the number of films available in a specific store and return the total via INOUT.

Tip: Use Stored Procedures, **INOUT** parameter

Input/Output data: Store ID (Input), filmTotal (Output)

Tables Used: inventory

Expected Output Columns: number of films (returned via INOUT)

Statements Used: COUNT(), WHERE, SET

Call the Procedure: Show how many films are available in store ID **1 and ID 2 separately**

```
DELIMITER $$
CREATE PROCEDURE GetFilmCountByStore (INOUT
store_id INT)
BEGIN
DECLARE filmTotal INT;
SELECT COUNT(*) INTO filmTotal
FROM inventory
WHERE inventory.store_id = store_id;
SET store_id = filmTotal;
END $$
DELIMITER ;
SET @store_id = 1;
CALL GetFilmCountByStore(@store_id);
SELECT @store_id AS filmTotal;
```

Task 3

GET POPULATION OF A REGION

Task 3: Return the total population of all countries in a given region using an INOUT parameter.

Tip: Use Stored Procedures, **INOUT** parameter

Input/Output: Region name (input), Total population (output)

Tables Used: country

Expected Output Columns: total population (via INOUT)

Statements Used: SUM(), WHERE, SET

Call the Procedure: Return the population of the region **'Southern Europe' and 'Central Africa' separately**

```
DELIMITER $$
CREATE PROCEDURE GetRegionPopulation(INOUT
region_name VARCHAR(50))
BEGIN
SELECT SUM(Population)
INTO region_name
FROM country
WHERE Region = region_name;
END $$
DELIMITER ;
SET @region = 'Southern Europe';
CALL GetRegionPopulation(@region);
SELECT @region AS region_population;
SET @region = 'Central Africa';
CALL GetRegionPopulation(@region);
SELECT @region AS region_population;
```